

背景

应用是业务能力承载者，所以在大型模型的世界里也有应用。大型模型的能力这里就不多说了，这里指的是基础大型模型的能力。比如大家熟悉的公有的 ChatGPT，开源可私有化部署的 qwen, chatglm 等。有了大型模型之后，大家会发现知识变得更加容易获取和使用。那怎么将这些技术能力融入到业务中来。

接下来，这里围绕大型模型来构建应用的时候使用的一个技术 RAG (Retrieval-Augmented Generation, 翻译成中文是检索增强生成) 来介绍 RAG 类型的应用。那为什么是 RAG 呢，我们可以看看下面的一段介绍，可能会对大家有所帮助。

Search-Ask 方法论：是一种用于问答系统的策略，结合了基于检索和基于生成的方法。它旨在充分利用检索和生成两种不同的技术，以提供更全面和准确的答案。在 Search-Ask 方法中，首先进行搜索 (Search) 阶段。用户的查询被发送到一个检索系统，该系统在预先建立的知识库或文档集合中执行搜索，并返回与查询相关的文档或候选答案。然后是询问 (Ask) 阶段。在此阶段，通过使用基于生成的方法，从检索到的文档或候选答案中提取或生成最终的答案。这可能涉及文本摘要、实体识别、关系抽取等技术来提取关键信息，或者使用生成模型 (如语言模型) 来生成具有上下文和语义连贯性的答案。通过结合搜索和生成两个阶段，Search-Ask 方法旨在通过检索提供初始的答案候选集，并通过生成进行进一步的筛选和精炼。这种方法的优势在于，检索阶段可以快速找到相关的文档或信息，而生成阶段可以通过理解上下文和语义生成更准确和丰富的答案。需要注意的是，具体实施 Search-Ask 方法的细节可以因应用场景和具体需求而有所不同。这种方法可以根据实际情况进行调整和定制，以达到最佳的问答效果。

概念

- **语料/知识库：**企业的一些知识材料。可以用于给大型模型的客户问答提供最贴近问题答案的上下文信息。基于语料可以给大型模型有一个清晰的上下文，回答的内容更加可控和准确。
- **Embeddings：**在自然语言处理中，Embeddings 通常指的是将单词或短语映射到连续向量空间中的向量表示。关键作用就是将语料/知识转变成另一种形式，方便后续更方便根据问题找到合适的一些参考材料。

- **向量数据库**：支持向量数据的存储和查询的数据库就可以理解成是向量数据库，目前也有很多基于原有的数据库实现了向量插件的数据库，比如 Postgresql，Qdrant，ElasticSearch 等等。语料/知识经过 Embeddings 之后就要保存的地方就是向量数据库。

- **检索器**：给大模型提供上下文是 RAG 中非常关键的能力。检索器主要就是从相关的语料存储的地方，通过不同技术的手段，拿到和本次问题最相关的上下文内容。可以理解成开卷考试带了小抄来的。检索器是在 ASK 的时候会用到的技术。根据用户的问题去向量数据库检索相关语料/知识作为问答的上下文给到大模型。当然这里的检索方案也是没有一个确定实现标准的，常见的有仅向量检索，也有混合检索（向量检索+关键字全文检索）等等。最终目标就是找到我们期望的和问题最相关的语料块/知识。

- **rerank（重排序）**：它的作用是对初步排序结果进行再次优化。Rerank 模型通常采用深度学习等技术，通过对用户的行为、兴趣等特征进行建模，从而更准确地预测用户的偏好，并对推荐结果进行重新排序。几乎差不多所有的 Embeddings 经过 rerank 处理之后，在命中率和 MRR（平均倒数排名）上都有了比较显著的提升。大模型对头部和尾部的输入更加偏重，如果准确答案在很居中的位置，回答的精准度就会稍微差一点。

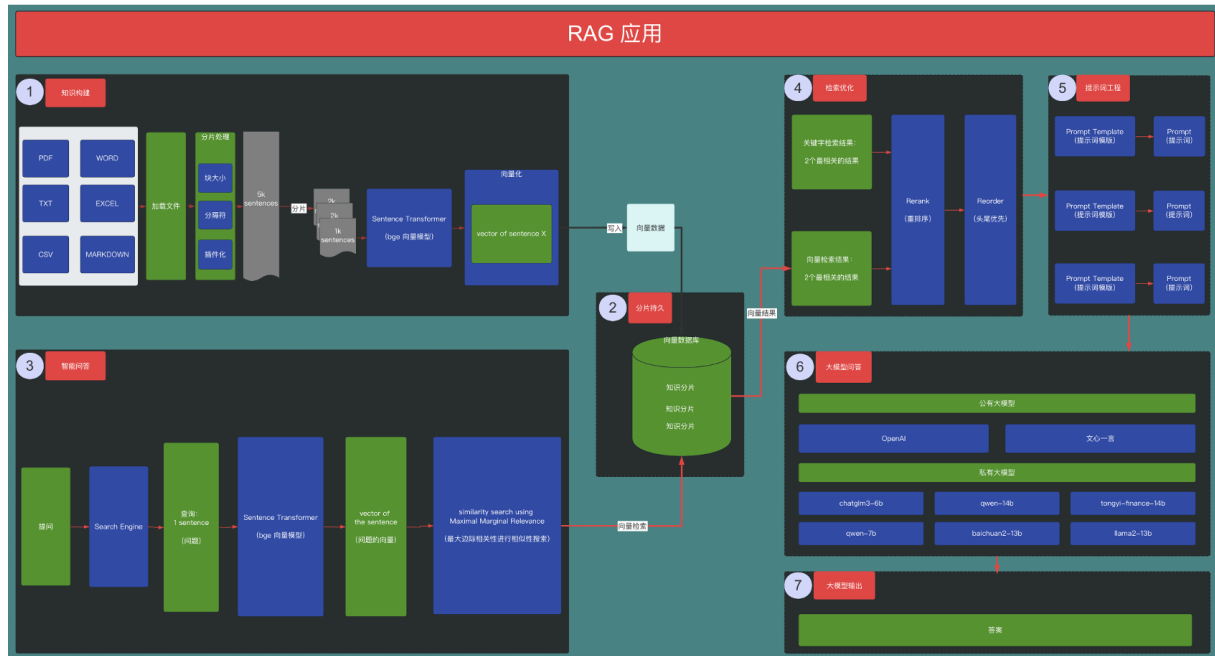
- **提示词**：这里简单的理解提示词就是给大模型的输入。包括上下文的信息，特定的系统指令，用户的问题等这些都可以理解为提示词的一部分。加起来就是一个完整的大模型的一次提问的输入。

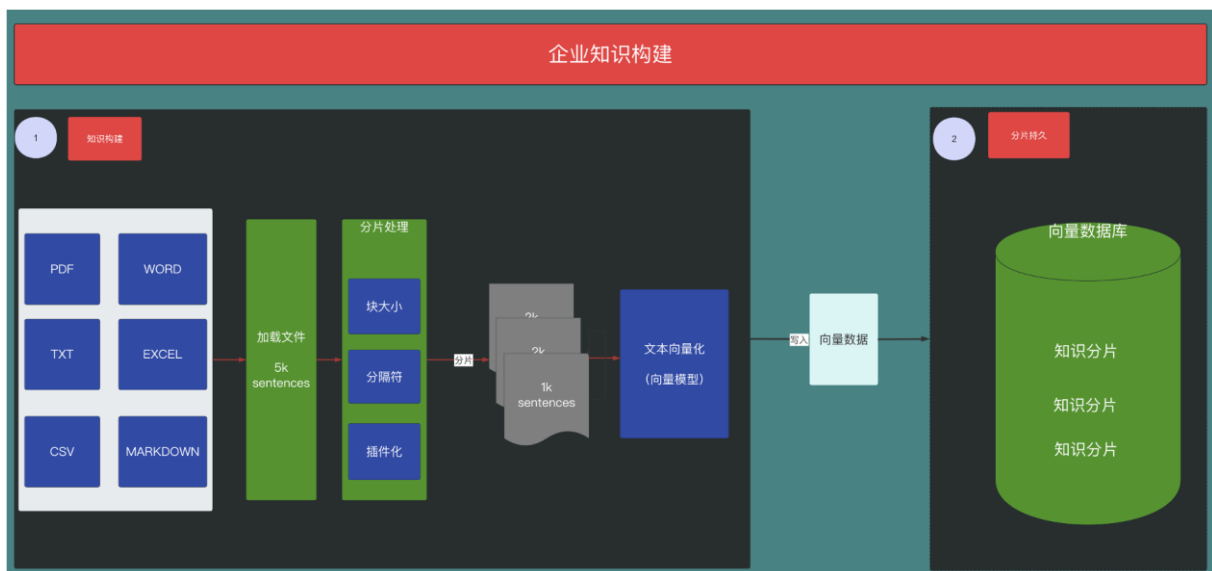
- **上下文**：指的是给大模型回答的时候一些背景信息，让大模型在回答的时候有一些前提的背景知识。比如基于一段文本回答一个特定于这段文本中相关的问题。比如历史对话消息，作为这次回答的背景信息，关联整个对话的过程，最终给出用户希望的答案。

- **LLM**：大语言模型。LLM 是一种基于深度学习的人工智能模型，具备强大的自然语言处理能力。它通过大规模的预训练过程，学习了广泛的语言知识和语言模式，并

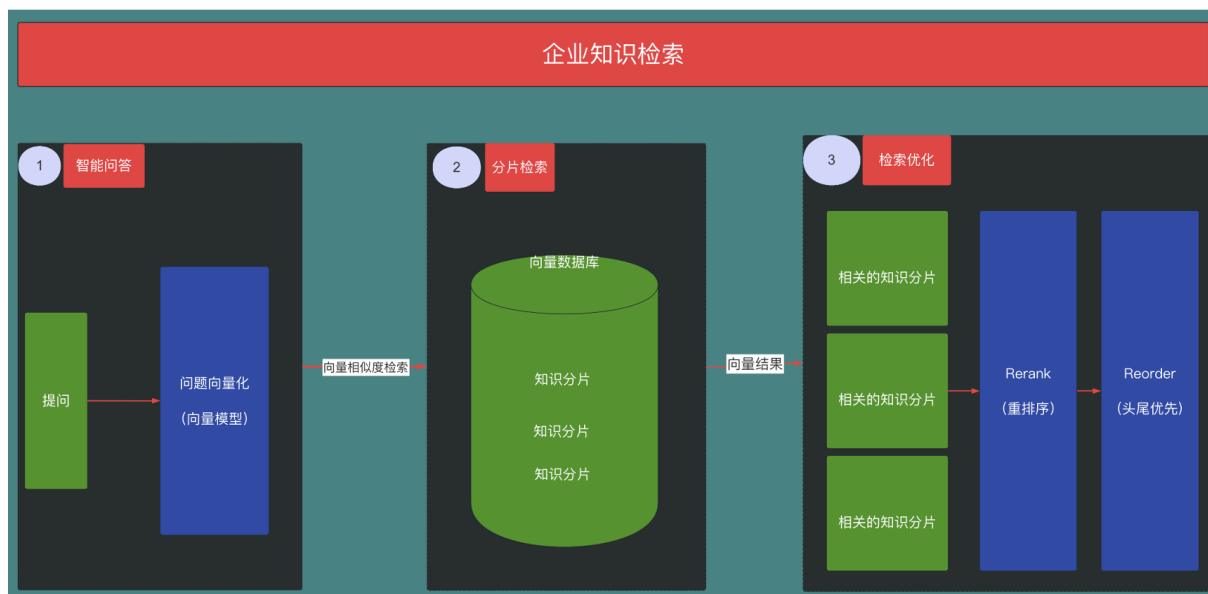
可以根据输入的文本生成连贯、合理的回复。

架构





1. 主要工作就是将语料文件经过文件拆分器拆分很多块，每一个块就是文件的一部分。不同的拆分策略会导致块的不同，比如包括按照字数来拆分，那到了指定的字数就会拆分出一个块；比如按照分隔符拆分，那发现到了指定分隔符的时候就会拆分出一个块处理；比如按照行来拆分，包括类似 excel, csv 文件这类的，可以按照一行就是一个块来处理。也可以根据需要将语料组织成需要的格式，比如问答对的方式，图文的方式，使用程序来按照需要来进行拆分的方式。选择哪种方式，也是根据情况而定的，最终的目标就是处理出适合当前场景的高质量语料出来。接下来，会将每一个块进行向量化的处理，这里的向量化的处理，指的是使用向量化的模型进行将文本处理成向量格式的数据。
 2. 接着上一步，处理出来的向量数据是需要保存起来。这里就会需要保存到向量数据库。至于是选择哪种向量数据库就看需求。当然也是综合考虑各方面的因素，包括是不是可以自主的很好的运维能力，包括是不是技术体系已经有了，包括是不是有一些向量数据库有一些必须的能力，比如有一些向量数据库只支持向量的能力，比如有一些向量数据库不仅仅包含了向量的能力，也包含全文检索的能力（类似 ES）。
- 基于语料构建出来的搜索引擎---检索：



1. 根据问题，将问题通过向量模型转成向量格式的数据。
2. 使用向量化的问题，去向量数据库进行相似检索，这里其实就是引入向量模型和向量数据库的关键点。因为要找出回答问题时用到的有用的相关知识的上下文。检索的时候也可以设置一些检索的参数，比如检索的相似度，比如检索出多少条最相似的语料块/知识。
3. 当检索出来了相关的语料块和知识之后，为了进一步提高语料块/知识和问题的相关性是更好的，需要经过一些处理提升相关能力以及更好的回答效果，可以使用 rerank（重排序）模型，根据问题和检索到的语料块/知识进行一次重拍得到更好的语料块/知识的排序结果。

● 基于大模型构建出来的问答系统：



1. 根据问题，检索出相关的语料块/知识。

2. 接下来，开始拼装提示词。提示词中，就会有一些部分是系统指令，有一些知识背景，有一些是用户问题。全部拼装起来之后就是给大模型的完整输入。
3. 接下来使用大模型的推理接口，传入完整的提示词和设置一些相关的推理接口的参数就可以正常的问答了。这里的参数指的就是常见的 temperature, top_p, stream 等等。

实战

- 语料块拆分和向量化处理：将业务数据进行安装拆分之后，使用向量模型将语料转成向量的数据形式。可以简单类比成在传统的基于 ES 技术中，对业务数据进行文档的分词和构建索引，来构建检索系统。这里的 sample 使用了 langchain 内置的 text split 来拆分语料。使用了 bge embedding 的模型来完成拆分后的语料块的向量化转换。这里使用了 langchain 的 `RecursiveCharacterTextSplitter` 的 `split_documents` 方法完成语料的拆分。使用了 langchain 的 `HuggingFaceEmbeddings` 的 `embed_documents` 来向量化语料。

```
import os
import time
from typing import List
import numpy as np
from tqdm import tqdm
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.document_loaders import PyPDFLoader
from langchain.embeddings import HuggingFaceEmbeddings

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=4000,
    chunk_overlap=100,
    length_function=len,
)

db_shards = 1

class VectorServiceDeployment:
    def __init__(self):
        # Load embedding model
        self.embeddings = HuggingFaceEmbeddings(model_name="/root/jina-embeddings-v2-base-zh", model_kwargs={'device': "cuda"})

    def embed_docs(self, shard):
        # Use embedding model to embed text chunks
        print(f"Starting process_shard of {len(shard)} chunks.")
        st = time.time()
        texts = [d.page_content for d in shard]
        response = self.embeddings.embed_documents(list(texts))
        et = time.time() - st
        print(f"Shard completed in {et} seconds.")
        return response

    def file_split(self):
        # Put your directory containing PDFs here
        directory = '/root/nowtest'
        pdf_documents = [os.path.join(directory, filename) for filename in os.listdir(directory)]
        langchain_documents = []
        for document in tqdm(pdf_documents):
            try:
                loader = PyPDFLoader(document)
                data = loader.load()
                langchain_documents.extend(data)
            except Exception:
                continue

        print("Splitting all documents")
        chunks = text_splitter.split_documents(langchain_documents)
        shards = np.array_split(chunks, db_shards)
        results = [self.process_shard(shards[i]) for i in range(db_shards)]
        return results
```

- 向量数据库操作

写入向量数据库：将向量化后的数据，写入到向量数据库中。目前有很多开源/商用的向量数据库，可选择性很大。这里的 sample 使用的是 pgvector (pg 的一个插件，可以将 pg 当做向量数据库来使用)。这里使用了 langchain 内置的 pgvector 的封装 PGVector，结合向量模型和拆分后的语料块，通过 `from_documents` 的方法完成向量入库。

```
import os
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.vectorstores.pgvector import PGVector
from langchain.vectorstores.pgvector import DistanceStrategy
from tqdm import tqdm
from langchain.document_loaders import PyPDFLoader
from langchain.embeddings import HuggingFaceEmbeddings

PGVECTOR_CONNECTION_STRING = PGVector.connection_string_from_db_params(
    driver=os.environ.get("PGVECTOR_DRIVER", "psycopg2"),
    host=os.environ.get("PGVECTOR_HOST", "10.xx.xx.xx"),
    port=int(os.environ.get("PGVECTOR_PORT", "5432")),
    database=os.environ.get("PGVECTOR_DATABASE", "xxx"),
    user=os.environ.get("PGVECTOR_USER", "xxx"),
    password=os.environ.get("PGVECTOR_PASSWORD", "xxx"),
)

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100,
    length_function=len,
)

# Put your directory containing PDFs here and split files
directory = '/root/known'
pdf_documents = [os.path.join(directory, filename) for filename in os.listdir(directory)]
langchain_documents = []
for document in tqdm(pdf_documents):
    try:
        loader = PyPDFLoader(document)
        data = loader.load()
        langchain_documents.extend(data)
    except Exception:
        continue

chunks = text_splitter.split_documents(langchain_documents)

#embed text chunks and insert into vector store
embeddings = HuggingFaceEmbeddings(model_name="/root/jina-embeddings-v2-base-zh" ,model_kwargs={'device': "cuda"})
db = PGVector.from_documents(
    embedding=embeddings,
    documents=chunks,
    collection_name="langchain_pg_collection01",
    connection_string=PGVECTOR_CONNECTION_STRING,
    distance_strategy=DistanceStrategy.COSINE,
    pre_delete_collection=False
)
```

查询向量数据：这里使用了 langchain 内置的 pgvector 的封装 PGVector 的相似度查询 `similarity_search_with_score` 方法完成相似语料块的查询，同时返回的结果会有对应的相似度 score，这个 score 的大小就可以分析哪个语料块是和用户问题的相似程度。


```

import os
from typing import List
from starlette.requests import Request
from langchain.vectorstores.pgvector import PGVector
from langchain.vectorstores.pgvector import DistanceStrategy
from langchain.embeddings import HuggingFaceEmbeddings

PGVECTOR_CONNECTION_STRING = PGVector.connection_string_from_db_params(
    driver=os.environ.get("PGVECTOR_DRIVER", "psycopg2"),
    host=os.environ.get("PGVECTOR_HOST", "10.xx.xx.xx"),
    port=int(os.environ.get("PGVECTOR_PORT", "5432")),
    database=os.environ.get("PGVECTOR_DATABASE", "xxx"),
    user=os.environ.get("PGVECTOR_USER", "xxx"),
    password=os.environ.get("PGVECTOR_PASSWORD", "xxx"),
)

class VectorSearchDeployment:
    def __init__(self):
        self.embeddings = HuggingFaceEmbeddings(model_name="/root/jina-embeddings-v2-base-zh", model_kwargs={'device': "cuda"})
        self.db = PGVector(
            connection_string=PGVECTOR_CONNECTION_STRING,
            embedding_function=self.embeddings,
            collection_name="langchain_pg_collection01",
            distance_strategy=DistanceStrategy.COSINE
        )

    def search(self, query):
        print(f"query is: {query}")
        search_results = self.db.similarity_search_with_score(query, k=6)
        context = ""
        for pack in search_results:
            doc, score = pack
            content = doc.page_content
            print(f"检索到的知识={s}, from={s}, score={s.f"% (content, doc.metadata.get("from"), score))
            context += content

        return context

```

- rerank 处理：这里使用 bge 的 rerank 模型，完成将用户问题，和经过相似度检索出来的语料块，再进一步进行重排序。每一次 `compute_score` 都会对用户问题和查出来的某一个语料块再次进行打分。经过 rerank 处理之后的语料块的顺序会对大模型来回答问题会有更好的精确度的提升。也是目前 RAG 标配的能力。很多商业/开源的方案里都支持了这个能力。

```

from FlagEmbedding import FlagReranker

#Load rerank model
reranker = FlagReranker(model_name_or_path="/tmp/bge-reranker-large")

# Prepare data
text_list = ["passage1", "passage2", "passage3", "passage4", "passage5",]
query = "question"

# Rerank data
rerank_results = {}
for corpus in text_list:
    pair = [query, corpus]
    score = reranker.compute_score(pair)
    rerank_results[corpus] = score

sorted_results = sorted(rerank_results.items(), key=lambda item: item[1], reverse=True)

```

- 组装提示词：提示词顾名思义就是最后大模型的完整输入。那在 RAG 中，重点是要给一个合理的上下文，以及用户的问题是什么，其次还会有一些约束大模型的系统指令。让大模型在约定的系统指令下，根据提高的上下文来回答用户的真实问题。从以下的 sample 中可以看到，会有系统指令（比如你不知道的时候就说不知道，不要随意编造），以及 context

就是相关的语料，query 就是用户的真实提问，最后组合成完整的提示词作为大模型的 input。这里使用的是 langchain 的提示词模版的 `format` 方法，将输入和模型处理成最后的提示词。

```
from langchain_core.prompts import PromptTemplate

template = """
<|SYSTEM|># chatglm3-6b
- You are a helpful, polite, fact-based agent for answering questions.
- Your answers include enough detail for someone to follow through on your suggestions.
<|USER|>
If you don't know the answer, just say that you don't know. Don't try to make up an answer.
Please answer the following query using the context provided.

CONTEXT:
{context}
=====
QUESTION: {query}
ANSWER: <|ASSISTANT|>"""
promptTemplate = PromptTemplate(template=template, input_variables=["context", "query"])

query = "question"
context = "knowledge..."

prompt = promptTemplate.format(query=query, context=context)
```

- 大模型推理：这里就是和大模型本身直接交互的地方。从使用的角度看，只要调用推理的 api 接口就可以了，对外是黑盒的。这里有一点要提一下，大模型对 RAG 应用的整体影响也是很大的。大模型的能力越强，RAG 应用的准确度和稳定性也是跟着水涨船高的。但是也不是就非得选择一些能力很强的公有的收费的模型。很多业务场景，依赖开源的小一点的模型，结合 RAG 的技术也有很好的效果。具体哪个行，哪个不行都不好说，都要根据实际场景来分析和测试。这里使用了 langchain 的 chain 机制，最后通过调用 chain 的 `invoke` 方法完成对话。

```

import getpass
import os
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

os.environ["OPENAI_API_KEY"] = getpass.getpass()
model = ChatOpenAI(model="gpt-3.5-turbo-0125")

template = """
- You are a helpful, polite, fact-based agent for answering questions.
- Your answers include enough detail for someone to follow through on your suggestions.

If you don't know the answer, just say that you don't know. Don't try to make up an answer.
Please answer the following query using the context provided.

CONTEXT:
{context}
=====
QUESTION: {query}
"""

prompt = ChatPromptTemplate.from_template(template)

chain = prompt | model | StrOutputParser()

chain.invoke({"query": "question", "context": "knowledge"})

```

总结

经过语料的处理，语料的向量化，语料的入库，语料的相似度检索就构成了一套基于向量技术的搜索引擎作为 RAG 应用的前半部分。接下来，使用提示词工程和大模型的能力，构建出下半部分。两部分结合就形成了完整的 RAG 应用。以上的 sample 的 code 的主要是帮助理解每个环节主要要做什么事情。在使用一些大模型应用的开发框架的时候，很多事情会更简单，可能没几行代码就做了很多事情了，比如知名度很高的 langchain。所以 RAG 本质上是一个方法论，同时已经有很多成熟的方案来实现了。

在 RAG 的整个流程中用到的这些技术都是相对独立的存在，所以最后的技术选型就看怎么选择了。比如向量数据库就有很多种，向量模型也有很多种，大语言模型也有很多种。